

Development Guide

잡필 프로젝트의 로컬 개발환경 실행 가이드입니다.

Repository를 Clone한 뒤 루트에서 `docker compose up` 을 실행하면
Frontend / Backend / PostgreSQL + pgvector 개발환경이 함께 실행됩니다.

1. 처음 실행

```
git clone https://github.com/Jinhyung01/career-compass.git
cd career-compass
docker compose up
```

정상 실행 후:

Service	Address
Frontend	http://localhost:5173
Backend Health	http://localhost:8080/actuator/health
PostgreSQL + pgvector	localhost:5432

백그라운드 실행:

```
docker compose up -d
```

2. 개발 중

Frontend

`frontend/` 의 Vue 코드를 수정하면 **자동으로 화면에 반영**됩니다.
별도의 Container 재시작은 필요하지 않습니다.

```
frontend/src/
```

Backend

`backend/` 의 Spring Boot 코드를 수정하면 **자동으로 재컴파일 및 재시작**되어 **변경사항이 반영**됩니다.
별도의 Container 재시작은 필요하지 않습니다.

```
backend/src/main/java/
```

3. 상태 확인 / 종료

현재 실행 상태 확인:

```
docker compose ps
```

종료:

```
docker compose down
```

4. 설정 또는 Dependency 변경

일반적인 소스 코드 수정은 Rebuild가 필요하지 않습니다.

`Dockerfile`, `build.gradle` 등 Image 또는 Backend Dependency 관련 설정을 변경한 경우에는 다시 Build합니다.

```
docker compose up --build
```

Frontend의 `package.json` 또는 `package-lock.json`을 변경한 경우 기존 `node_modules` Volume에 Dependency를 다시 설치합니다.

```
docker compose run --rm frontend npm ci
docker compose up --build
```

`docker-compose.yml`을 변경한 경우에는 다시 실행합니다.

```
docker compose up
```

5. Log 확인

전체:

```
docker compose logs -f
```

Frontend:

```
docker compose logs -f frontend
```

Backend:

```
docker compose logs -f backend
```

Database:

```
docker compose logs -f db
```

6. Database Schema 변경

DB Schema는 **Flyway Migration**으로 관리합니다.

Migration 위치:

```
backend/src/main/resources/db/migration/
```

예:

```
v1__create_initial_schema.sql
v2__add_company_column.sql
v3__create_report_table.sql
```

이미 `main`에 Merge된 Migration 파일은 수정하지 않습니다.

```
v1 수정 ❌
v2 새로 추가 ✅
```

새 Migration 파일을 추가한 후 Backend를 재시작합니다.

```
docker compose restart backend
```

Backend 시작 시 Flyway가 아직 적용되지 않은 Migration을 자동으로 적용합니다.

적용 결과 확인:

```
docker compose logs -f backend
```

일반적인 Schema 변경 때문에 DB Volume을 삭제할 필요는 없습니다.

7. PostgreSQL / Vector Data

현재 Vector DB는 별도의 서버가 아니라 **PostgreSQL에 pgvector Extension을 적용한 구조**입니다.

```
PostgreSQL
├─ 일반 서비스 데이터
├─ profile_content.embedding (VECTOR(1536))
└─ company_insight.embedding (VECTOR(1536), HNSW Index)
```

따라서 일반 데이터와 Vector 데이터는 같은 PostgreSQL Container에 저장됩니다.

Mock 데이터

초기 개발 단계에서는 실제 AI 호출보다 **Mock 데이터로 FE-BE 흐름을 먼저 완성**합니다.

회원, 기업, 현직자 인터뷰 정보 등 공통으로 필요한 Mock 데이터는
Flyway Seed Migration으로 관리합니다.

위치 예:

```
backend/src/main/resources/db/migration/V2__insert_mock_data.sql
```

Repository를 처음 Clone한 팀원이 `docker compose up`을 실행하면
Backend 시작 시 Flyway가 Schema Migration 다음 Seed Migration을 자동 적용합니다.

```
docker compose up
```

Seed Migration은 각 DB Volume에 한 번만 적용됩니다.

`main`에 Merge된 Seed Migration은 수정하지 않고, 변경이 필요하면 새 Version Migration을 추가합니다.

Embedding 데이터

`VECTOR(1536)` 값은 직접 작성하지 않습니다.

Vector 검색 기능을 구현할 때 다음 흐름으로 생성합니다.

```
현직자 인터뷰 Text
    ↓
Embedding API
    ↓
1536차원 Vector
    ↓
company_insight.embedding 저장
```

즉 개발 순서는 다음과 같습니다.

1. Mock 데이터로 FE-BE 기능 개발
2. 현직자 인터뷰 Text 데이터 준비
3. Embedding 생성 Script 또는 Backend 로직 구현
4. 생성된 Vector를 PostgreSQL(pgvector)에 저장
5. Vector 유사도 검색 연결

Embedding 모델을 변경할 경우 출력 Vector 차원이 현재 DB의 `VECTOR(1536)` 과 일치하는지 반드시 확인합니다.

8. Database 초기화

일반 종료:

```
docker compose down
```

이 경우 로컬 DB 데이터는 유지됩니다.

DB 데이터를 모두 삭제하고 처음 상태로 다시 시작하려면:

```
docker compose down -v
docker compose up
```

`-v` 사용 시 PostgreSQL 데이터와 Migration 정보뿐 아니라

Frontend `node_modules`, Backend Gradle Cache 등 Compose의 모든 Named Volume이 삭제됩니다.

다시 실행하면 빈 DB에 Flyway Migration이 처음부터 적용됩니다.

9. 환경변수

기본 개발환경은 별도의 `.env` 없이 실행할 수 있도록 구성합니다.

OpenAI API를 사용하는 경우:

```
cp .env.example .env
```

`.env` 에 발급받은 API Key를 입력합니다.

```
OPENAI_API_KEY=your_api_key
```

`.env` 는 Git에 Commit하지 않습니다.

OpenAI API 연동 전에는 Mock 데이터를 사용하므로
API Key 없이도 기본 개발환경은 실행할 수 있습니다.

10. 문제가 있을 때

서비스 상태 확인

```
docker compose ps
```

Log 확인

```
docker compose logs -f
```

또는 문제가 있는 서비스만 확인합니다.

```
docker compose logs -f frontend
docker compose logs -f backend
docker compose logs -f db
```

Dependency / Docker 설정 변경 후 문제가 생긴 경우

```
docker compose up --build
```

Frontend Dependency를 변경한 경우:

```
docker compose run --rm frontend npm ci
docker compose up --build
```

PostgreSQL 5432 Port가 이미 사용 중인 경우

`.env.example` 을 복사해 `.env` 를 만들고 Host에 공개할 Port만 변경합니다.

```
POSTGRES_PORT=5433
```

Backend의 Container 내부 DB 연결은 계속 `db:5432` 를 사용합니다.

새 Migration이 적용되지 않는 경우

```
docker compose restart backend
docker compose logs -f backend
```

DB를 완전히 초기화해야 하는 경우

```
docker compose down -v
docker compose up
```

Quick Reference

실행

```
docker compose up
```

백그라운드 실행

```
docker compose up -d
```

상태 확인

```
docker compose ps
```

로그 확인

```
docker compose logs -f
```

종료

```
docker compose down
```

Image / Dependency 변경 후 Rebuild

```
docker compose up --build
```

Frontend Dependency 변경

```
docker compose run --rm frontend npm ci
```

```
docker compose up --build
```

새 Flyway Migration 적용

```
docker compose restart backend
```

Mock 데이터는 Flyway Seed Migration으로 자동 적용

```
# backend/src/main/resources/db/migration/V2__insert_mock_data.sql
```

DB까지 완전 초기화

```
docker compose down -v
```

```
docker compose up
```